WEEK 10

PROBLEM STATEMENT:

Advanced Problem-Solving Approach using linear and non-linear collections.
An online shopping system must maintain products, quickly search products by ID, and process orders according to priority.

CODE:

```
import java.util.*;

class Main {
public static void main(String[] args) {
Scanner sc = new Scanner(System.in);

   ArrayList<String> products = new ArrayList<>();

   HashMap<Integer, String> productMap = new HashMap<>();

   System.out.print("Enter number of products: ");

   int n = sc.nextInt();

   for (int i = 0; i < n; i++) {

      System.out.print("Enter product ID and name: ");

      int id = sc.nextInt();

      String name = sc.next();

      products.add(name);

      productMap.put(id, name);

   }

   System.out.println("Products: " + products);

   System.out.print("Enter product ID to search: ");

   int id = sc.nextInt();

   if (productMap.containsKey(id)) {

      System.out.println("Product found: " + productMap.get(id));

   } else {

      System.out.println("Product not found");

   }

   PriorityQueue<Integer> orders = new PriorityQueue<>();
```

```
    System.out.print("Enter number of orders: ");
    int m = sc.nextInt();
    for (int i = 0; i < m; i++) {
        System.out.print("Enter order priority: ");
        orders.add(sc.nextInt());
    }
    System.out.println("Order processing:");
    while (!orders.isEmpty()) {
        System.out.println("Priority: " + orders.poll());
    }
    sc.close();
}
}
```

INPUT:

3

101 Laptop

102 Mobile

103 Tablet

102

3

3

1

2

OUTPUT:

Products: [Laptop, Mobile, Tablet]

Product found: Mobile

Order processing:

Priority: 1

Priority: 2

Priority: 3

PROBLEM STATEMENT:

Student Marks Management System:

A teacher wants to create a Java program to store the marks of students in a class. The number of students is not known in advance, so the program should use a dynamic array. The teacher can add marks one by one, and the array should automatically increase its size when it becomes full.

CODE:

```
import java.util.*;

class Main {
public static void main(String[] args) {
Scanner sc = new Scanner(System.in);

   ArrayList<Integer> marks = new ArrayList<>();

   System.out.print("Enter number of students: ");

   int n = sc.nextInt();

   for (int i = 0; i < n; i++) {

      System.out.print("Enter marks: ");

      int mark = sc.nextInt();

      marks.add(mark);

   }

   System.out.println("Student Marks: " + marks);

   sc.close();

}

}
```

INPUT:

Enter number of students: 3

Enter marks: 96

Enter marks: 7

Enter marks: 55

OUTPUT:

Student Marks: [96, 7, 55]

PROBLEM STATEMENT:

Browser History — Linear Collection

A web browser stores recently visited pages. When the user presses the Back button, the most recently visited page should be removed first.

CODE:

```
import java.util.*;

class Main {
public static void main(String[] args) {
Scanner sc = new Scanner(System.in);
    Stack<String> history = new Stack<>();
    System.out.print("Enter number of pages: ");
    int n = sc.nextInt();
    for (int i = 0; i < n; i++) {
        System.out.print("Enter page name: ");
        history.push(sc.next());
    }
    System.out.println("Browser History: " + history);
    System.out.println("Press Back button");
    if (!history.isEmpty()) {
        history.pop();
    }
    System.out.println("After Back: " + history);
    sc.close();
}
}
```

INPUT:

```
Enter number of pages: 5
Enter page name: A
Enter page name: B
Enter page name: C
```

Enter page name: X

Enter page name: Y

OUTPUT:

Browser History: [A, B, C, X, Y]

Press Back button

After Back: [A, B, C, X]

PROBLEM STATEMNT:

Student Marks Management — Linear Collection

A teacher does not know the number of students in advance. Student marks need to be stored dynamically.

Question

Write a Java program using an ArrayList to:

- Store marks.
- Display marks.
- Find the highest mark.
- Calculate the average

CODE:

```
import java.util.*;

class Main {
public static void main(String[] args) {
Scanner sc = new Scanner(System.in);

   ArrayList<Integer> marks = new ArrayList<>();

   System.out.print("Enter number of students: ");

   int n = sc.nextInt();

   int sum = 0;

   int highest = 0;

   for (int i = 0; i < n; i++) {

      System.out.print("Enter marks: ");

      int mark = sc.nextInt();

      marks.add(mark);
```

```
        sum = sum + mark;
        if (i == 0 || mark > highest) {
            highest = mark;
        }
    }
    System.out.println("Student Marks: " + marks);
    System.out.println("Highest Mark: " + highest);
    double average = (double) sum / n;
    System.out.println("Average Mark: " + average);
    sc.close();
}
}
```

INPUT:

Enter number of students: 3

Enter marks: 45

Enter marks: 7

Enter marks: 31

OUTPUT:

Student Marks: [45, 7, 31]

Highest Mark: 45

Average Mark: 27.666666666666668

PROBLEM STATEMENT:

Music Playlist — Singly Linked List

A music application maintains songs in a playlist. Users can add songs to the playlist and remove a song when they no longer want it.

Question

Implement a linked list playlist that supports:

1. Adding songs.
2. Removing a song.

3. Displaying the playlist.

CODE:

```
import java.util.Scanner;

class Main {
static class Node {
String song;
Node next;

   Node(String song) {

      this.song = song;

      this.next = null;

   }

}

static Node head = null;

static void addSong(String song) {

   Node newNode = new Node(song);

   if (head == null) {

      head = newNode;

   } else {

      Node temp = head;

      while (temp.next != null) {

         temp = temp.next;

      }

      temp.next = newNode;

   }

}

static void removeSong(String song) {

   if (head == null) {

      return;

   }

   if (head.song.equals(song)) {
```

```
            head = head.next;
            return;
        }
        Node temp = head;
        while (temp.next != null && !temp.next.song.equals(song)) {
            temp = temp.next;
        }
        if (temp.next != null) {
            temp.next = temp.next.next;
        } else {
            System.out.println("Song not found");
        }
    }
    static void display() {
        Node temp = head;
        while (temp != null) {
            System.out.print(temp.song + " -> ");
            temp = temp.next;
        }
        System.out.println("NULL");
    }
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter number of songs: ");
        int n = sc.nextInt();
        for (int i = 0; i < n; i++) {
            System.out.print("Enter song name: ");
            addSong(sc.next());
        }
```

```
    System.out.println("Original Playlist:");
    display();
    System.out.print("Enter song to remove: ");
    String song = sc.next();
    removeSong(song);
    System.out.println("Updated Playlist:");
    display();
    sc.close();
}
}
```

INPUT:

Enter number of songs: 3

Enter song name: X

Enter song name: Y

Enter song name: Z

OUTPUT:

Original Playlist:

X -> Y -> Z -> NULL

Enter song to remove: Y

Updated Playlist:

X -> Z -> NULL

PROBLEM STATEMENT:

Find All Unique Items Purchased

A customer purchases several items, including repeated items. Write a Java program using a LinkedHashSet to display only unique items in the order they were purchased.

CODE:

```
import java.util.*;
```

```
class Main {
   public static void main(String[] args) {
      Scanner sc = new Scanner(System.in);
      LinkedHashSet<String> items = new LinkedHashSet<>();
      System.out.print("Enter number of items: ");
      int n = sc.nextInt();
      for (int i = 0; i < n; i++) {
         items.add(sc.next());
      }
      System.out.println("Unique items: " + items);
      sc.close();
   }
}
```

INPUT:

Enter number of items: 5

BAT

BALL

STUMPS

BAT

PLAYER

OUTPUT:

Unique items: [BAT, BALL, STUMPS, PLAYER]

PROBLEM STATEMENT:

Display Numbers in Sorted Order

Scenario: A teacher enters students' marks, but some marks are repeated. Write a Java program using a TreeSet to display the unique marks in ascending order.

CODE:

```
import java.util.*;
```

```
class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        TreeSet<Integer> marks = new TreeSet<>();
        System.out.println("Enter number of marks:");
        int n = sc.nextInt();
        System.out.println("Enter the marks:");
        for (int i = 0; i < n; i++) {
            marks.add(sc.nextInt());
        }
        System.out.println("Sorted unique marks:");
        System.out.println(marks);
        sc.close();
    }
}
```

INPUT:

Enter number of marks:

3

Enter the marks:

25

66

7

OUTPUT:

Sorted unique marks:

[7, 25, 66]

PROBLEM STATEMENT:

Find Common Subjects Between Two Students

Two students have selected different subjects. Write a Java program using retainAll() to find the subjects common to both.

CODE:

```
import java.util.*;
class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        ArrayList<String> student1 = new ArrayList<>();
        ArrayList<String> student2 = new ArrayList<>();
        System.out.println("Enter number of subjects for Student 1:");
        int n1 = sc.nextInt();
        System.out.println("Enter subjects:");
        for (int i = 0; i < n1; i++) {
            student1.add(sc.next());
        }
        System.out.println("Enter number of subjects for Student 2:");
        int n2 = sc.nextInt();
        System.out.println("Enter subjects:");
        for (int i = 0; i < n2; i++) {
            student2.add(sc.next());
        }
        student1.retainAll(student2);
        System.out.println("Common subjects: " + student1);
        sc.close();
    }
}
```

INPUT:

```
Enter number of subjects for Student 1:
2
Enter subjects:
java
```

python

Enter number of subjects for Student 2:

2

Enter subjects:

java

c

OUTPUT:

Common subjects: [java]

PROBLEM STATEMENT:

Phone Contact Directory

A mobile application stores people's names and phone numbers. Users can search for a contact or update an existing phone number

CODE:

```
import java.util.*;

class Main {
   public static void main(String[] args) {
      Scanner sc = new Scanner(System.in);
      HashMap<String, String> contacts = new HashMap<>();
      System.out.println("Enter number of contacts:");
      int n = sc.nextInt();
      for (int i = 0; i < n; i++) {
         System.out.println("Enter name and phone number:");
         String name = sc.next();
         String phone = sc.next();
         contacts.put(name, phone);
      }
      System.out.println("Enter name to search:");
      String search = sc.next();
      if (contacts.containsKey(search)) {
```

```
            System.out.println("Phone number: " + contacts.get(search));
            System.out.println("Enter new phone number:");
            String newPhone = sc.next();
            contacts.put(search, newPhone);
            System.out.println("Contact updated successfully!");
            System.out.println("Updated phone number: " + contacts.get(search));
        } else {
            System.out.println("Contact not found!");
        }
        sc.close();
    }
}
```

INPUT:

Enter number of contacts:

2

Enter name and phone number:

Karthik 9876543210

Enter name and phone number:

Priya 9123456780

Enter name to search:

Karthik

Enter new phone number:

9988776655

OUTPUT:

Phone number: 9876543210

Contact updated successfully!

Updated phone number: 9988776655

WEEK 11

PROBLEM STATEMENT:

Problem

Given an array nums of size n, return the majority element.

The majority element is the element that appears more than ⌊n / 2⌋ times. You may assume that the majority element always exists in the array.

Example 1:

- Input: nums = [3,2,3]
- Output: 3

Example 2:

- Input: nums = [2,2,1,1,1,2,2]
- Output: 2

Constraints:

- n == nums.length
- 1 <= n <= 5 * 104
- -109 <= nums[i] <= 109
- The input is generated such that a majority element will exist in the array.

CODE:

```
import java.util.*;

class Main {
public static void main(String[] args) {
int[] nums = {2, 2, 1, 1, 1, 2, 2};

   HashMap<Integer, Integer> map = new HashMap<>();

   for (int num : nums) {

      map.put(num, map.getOrDefault(num, 0) + 1);

      if (map.get(num) > nums.length / 2) {

         System.out.println(num);

         return;

      }

   }
```

}

}

INPUT:

nums = [2,2,1,1,1,2,2]

OUTPUT:

2

PRIBLEM STATEMENT:

Problem

Given a string s, find the **first** non-repeating character in it and return its index. If it **does not** exist, return -1.

Example 1:

Input: s = "leetcode"

Output: 0

Explanation:

The character 'l' at index 0 is the first character that does not occur at any other index.

Example 2:

Input: s = "loveleetcode"

Output: 2

Example 3:

Input: s = "aabb"

Output: -1

Constraints:

- $1 <= s.length <= 10^5$
- s consists of only lowercase English letters.

CODE:

```
class Solution {
public int firstUniqChar(String s) {
for (int i = 0; i < s.length(); i++) {
int count = 0;
        for (int j = 0; j < s.length(); j++) {
```

```
            if (s.charAt(i) == s.charAt(j)) {
                count++;
            }
        }
        if (count == 1) {
            return i;
        }
    }
    return -1;
}
}
```

INPUT:

s = "leetcode"

OUTPUT:

0

PROBLEM STATEMENT:

Problem

Given a non-empty array of non-negative integers nums, the degree of this array is defined as the maximum frequency of any one of its elements.

Your task is to find the smallest possible length of a (contiguous) subarray of nums, that has the same degree as nums.

CODE:

```
import java.util.Scanner;

class Main {
public static void main(String[] args) {
Scanner sc = new Scanner(System.in);

    int n = sc.nextInt();

    int[] nums = new int[n];

    for (int i = 0; i < n; i++) {

        nums[i] = sc.nextInt();
```

```
    }
    int max = 0;
    int minLength = n;
    for (int i = 0; i < n; i++) {
        int count = 0;
        int first = -1, last = -1;
        for (int j = 0; j < n; j++) {
            if (nums[i] == nums[j]) {
                count++;
                if (first == -1) {
                    first = j;
                }
                last = j;
            }
        }
        if (count > max) {
            max = count;
            minLength = last - first + 1;
        } else if (count == max) {
            minLength = Math.min(minLength, last - first + 1);
        }
    }
    System.out.println(minLength);
    sc.close();
}
}
```

INPUT:

7

1 2 2 3 1 4 2

OUTPUT:

6

PROBLEM STATEMENT:

Problem

Given the names and grades for each student in a class of students, store them in a nested list and print the name(s) of any student(s) having the second lowest grade.

Note: If there are multiple students with the second lowest grade, order their names alphabetically and print each name on a new line.

Example

The ordered list of scores is, so the second lowest score is. There are two students with that score: Ordered alphabetically, the names are printed as:

alpha

beta

Input Format

The first line contains an integer, , the number of students.
The subsequent lines describe each student over lines.
- The first line contains a student's name.
- The second line contains their grade.

Constraints

- There will always be one or more students having the second lowest grade.

Output Format

Print the name(s) of any student(s) having the second lowest grade in. If there are multiple students, order their names alphabetically and print each one on a new line.

CODE:

```
import java.util.*;

class Main {
public static void main(String[] args) {
Scanner sc = new Scanner(System.in);

   int n = sc.nextInt();

   String[] names = new String[n];

   double[] grades = new double[n];
```

```
    for (int i = 0; i < n; i++) {
        names[i] = sc.next();
        grades[i] = sc.nextDouble();
    }
    double lowest = Double.MAX_VALUE;
    double secondLowest = Double.MAX_VALUE;
    for (int i = 0; i < n; i++) {
        if (grades[i] < lowest) {
            secondLowest = lowest;
            lowest = grades[i];
        } else if (grades[i] > lowest && grades[i] < secondLowest) {
            secondLowest = grades[i];
        }
    }
    ArrayList<String> result = new ArrayList<>();
    for (int i = 0; i < n; i++) {
        if (grades[i] == secondLowest) {
            result.add(names[i]);
        }
    }
    Collections.sort(result);
    for (String name : result) {
        System.out.println(name);
    }
    sc.close();
}
}
```

INPUT:

5

Harry

37.21

Berry

37.21

Tina

37.2

Akriti

41

Harsh

39

OUTPUT:

Berry

Harry

PROBLEM STATEMENT:

Problem

You are given lines. In each line there are zero or more integers. You need to answer a few queries where you need to tell the number located in position of line.

Take your input from System.in.

Input Format
The first line has an integer . In each of the next lines there will be an integer denoting number of integers on that line and then there will be space-separated integers. In the next line there will be an integer denoting number of queries. Each query will consist of two integers and .

- Each number will fit in signed integer.
- Total number of integers in lines will not cross .

Output Format
In each line, output the number located in yth position of xth line. If there is no such position, just print "ERROR!"

CODE:

```
import java.util.*;
```

```
class Main {
public static void main(String[] args) {
Scanner sc = new Scanner(System.in);
    int n = sc.nextInt();
    ArrayList<ArrayList<Integer>> list = new ArrayList<>();
    for (int i = 0; i < n; i++) {
        int d = sc.nextInt();
        ArrayList<Integer> row = new ArrayList<>();
        for (int j = 0; j < d; j++) {
            row.add(sc.nextInt());
        }
        list.add(row);
    }
    int q = sc.nextInt();
    for (int i = 0; i < q; i++) {
        int x = sc.nextInt();
        int y = sc.nextInt();
        if (x >= 1 && x <= n && y >= 1 && y <= list.get(x - 1).size()) {
            System.out.println(list.get(x - 1).get(y - 1));
        } else {
            System.out.println("ERROR!");
        }
    }
    sc.close();
}
}
```

INPUT:

```
5
5 41 77 74 22 44
1 12
```

4 37 34 36 52

0

3 20 22 33

5

1 3

3 4

3 1

4 3

5 2

OUTPUT:

74

52

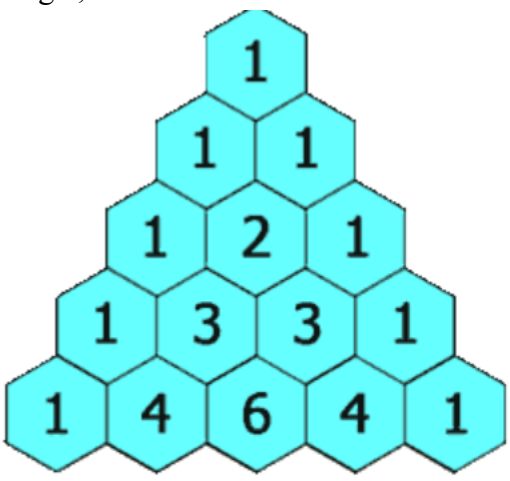
37

ERROR!

22

PROBLEM STATEMENT:

Given an integer numRows, return the first numRows of Pascal's triangle. In Pascal's triangle, each number is the sum of the two numbers directly above it as shown:



Example 1:

Input: numRows = 5

Output: [[1],[1,1],[1,2,1],[1,3,3,1],[1,4,6,4,1]]

Example 2

Input: numRows = 1

Output: [[1]]

Constraints:

- 1 <= numRows <= 30

CODE:

```
import java.util.*;

class Main {
public static void main(String[] args) {
Scanner sc = new Scanner(System.in);
   int numRows = sc.nextInt();
   ArrayList<ArrayList<Integer>> triangle = new ArrayList<>();
   for (int i = 0; i < numRows; i++) {
      ArrayList<Integer> row = new ArrayList<>();
      for (int j = 0; j <= i; j++) {
         if (j == 0 || j == i) {
            row.add(1);
         } else {
            row.add(triangle.get(i - 1).get(j - 1)
                  + triangle.get(i - 1).get(j));
         }
      }
      triangle.add(row);
   }
   System.out.println(triangle);
   sc.close();
}
}
```

INPUT:

5

OUTPUT:

[[1], [1, 1], [1, 2, 1], [1, 3, 3, 1], [1, 4, 6, 4, 1]]

PROBLEM STATEMENT:

Problem

Complete the function in the editor below, which has parameter: a pointer to the root of a binary tree. It must print the values in the tree's preorder traversal as a single line of space-separated values.

Input Format

Our test code passes the root node of a binary tree to the *preOrder* function.

Constraints

Nodes in the tree

Output Format

Print the tree's preorder traversal as a single line of space-separated values.

CODE:

```
import java.util.*;

class Node {
int data;
Node left, right;

Node(int data) {

   this.data = data;

}

}

class Main {
static void preOrder(Node root) {
if (root == null) {
return;
}

   System.out.print(root.data + " ");

   preOrder(root.left);

   preOrder(root.right);

}

public static void main(String[] args) {
```

```
    Node root = new Node(1);
    root.left = new Node(2);
    root.right = new Node(3);
    root.left.left = new Node(4);
    root.left.right = new Node(5);
    preOrder(root);
}
}
```

OUTPUT:

1 2 4 5 3

PROBLEM STATEMENT:

Problem

Find two numbers in an array whose sum equals a target value.

Example

Input: nums = [2, 7, 11, 15]

Target = 9

Output: [0, 1]

CODE:

```
class Main {
public static void main(String[] args) {
int[] nums = {2, 7, 11, 15};
int target = 9;
    for (int i = 0; i < nums.length; i++) {
        for (int j = i + 1; j < nums.length; j++) {
            if (nums[i] + nums[j] == target) {
                System.out.println("[" + i + ", " + j + "]");
                return;
            }
        }
    }
```

}

}

INPUT:

nums = [2, 7, 11, 15]

OUTPUT:

[0,1]

PROBLEM STATEMENT:

Problem Description

A smart city application stores road connectivity information using nested collections. Given city junctions and roads, determine whether a route exists between two junctions using graph representation with collections.

Input Format

- First line contains integers N and M.
- Next M lines contain connected junction pairs.
- Last line contains source and destination.

Output Format

Print YES if route exists, otherwise NO.

Constraints

- $1 \leq N \leq 10^5$
- $1 \leq M \leq 2 \times 10^5$

Sample Input

5 4
1 2
2 3
3 4
4 5
1 5

Sample Output

YES

CODE:

```
import java.util.*;
```

```
class Main {
public static void main(String[] args) {
Scanner sc = new Scanner(System.in);
    int n = sc.nextInt();
    int m = sc.nextInt();
    ArrayList<ArrayList<Integer>> graph = new ArrayList<>();
    for (int i = 0; i <= n; i++) {
        graph.add(new ArrayList<>());
    }
    for (int i = 0; i < m; i++) {
        int u = sc.nextInt();
        int v = sc.nextInt();
        graph.get(u).add(v);
        graph.get(v).add(u);
    }
    int source = sc.nextInt();
    int destination = sc.nextInt();
    boolean[] visited = new boolean[n + 1];
    Queue<Integer> queue = new LinkedList<>();
    queue.add(source);
    visited[source] = true;
    while (!queue.isEmpty()) {
        int current = queue.poll();
        if (current == destination) {
            System.out.println("YES");
            sc.close();
            return;
        }
        for (int next : graph.get(current)) {
            if (!visited[next]) {
```

```
                visited[next] = true;
                queue.add(next);
            }
        }
    }
    System.out.println("NO");
    sc.close();
}
}
```

INPUT:

5 4

1 2

2 3

3 4

4 5

1 5

OUTPUT:

YES

WEEK 12

PROBLEM STATEMENT:

– Hashtag Frequency Counter

Problem

A social media platform receives N hashtags. Count the frequency of each hashtag and display the hashtags along with their frequencies.

If multiple hashtags have the same frequency, display them in lexicographical order.

CODE:

```
import java.util.*;
class Main {
    public static void main(String[] args) {
```

```
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();
        Map<String, Integer> map = new HashMap<>();
        for (int i = 0; i < n; i++) {
            String s = sc.next();
            map.put(s, map.getOrDefault(s, 0) + 1);
        }
        List<String> list = new ArrayList<>(map.keySet());
        Collections.sort(list, (a, b) -> {
            if (map.get(a).equals(map.get(b))) {
                return a.compareTo(b);
            }
            return map.get(b) - map.get(a);
        });
        for (String s : list) {
            System.out.println(s + " " + map.get(s));
        }
        sc.close();
    }
}
```

INPUT:

5

java

python

java

c

python

OUTPUT:

Java 2

Python 2

c 1

PROBLEM STATEMENT:

Problem

You are given a collection of strings followed by a collection of query strings. For every query, determine how many times it occurs in the original collection.

For large input, the solution should avoid scanning the entire collection for every query.

CODE:

```
import java.util.*;

class Main {
   public static void main(String[] args) {
      Scanner sc = new Scanner(System.in);
      int n = sc.nextInt();
      String[] arr = new String[n];
      Map<String, Integer> map = new HashMap<>();
      for (int i = 0; i < n; i++) {
         arr[i] = sc.next();
         map.put(arr[i], map.getOrDefault(arr[i], 0) + 1);
      }
      int q = sc.nextInt();
      for (int i = 0; i < q; i++) {
         String s = sc.next();
         System.out.println(map.getOrDefault(s, 0));
      }
      sc.close();
   }
}
```

INPUT:

4

aba

baba

aba

xzxb

3

aba

xzxb

ab

OUTPUT:

2

1

0

PROBLEM STATEMENT:

Given an integer array nums and an integer k, return *the* k *most frequent elements*. You may return the answer in any order.

Constraints:

- $1 <= nums.length <= 10^5$
- $-10^4 <= nums[i] <= 10^4$
- k is in the range [1, the number of unique elements in the array].
- It is guaranteed that the answer is **unique**.

CODE:

```
import java.util.*;
class Solution {
    public int[] topKFrequent(int[] nums, int k) {
        HashMap<Integer, Integer> map = new HashMap<>();
        for (int num : nums) {
            map.put(num, map.getOrDefault(num, 0) + 1);
        }
        List<Integer>[] buckets = new List[nums.length + 1];
```

```
        for (int num : map.keySet()) {
            int frequency = map.get(num);
            if (buckets[frequency] == null) {
                buckets[frequency] = new ArrayList<>();
            }
            buckets[frequency].add(num);
        }
        int[] result = new int[k];
        int index = 0;
        for (int frequency = nums.length; frequency >= 1; frequency--) {
            if (buckets[frequency] != null) {
                for (int num : buckets[frequency]) {
                    result[index++] = num;
                    if (index == k) {
                        return result;
                    }
                }
            }
        }
        return result;
    }
}
```

INPUT:

nums = [1,1,1,2,2,3], k = 2

OUTPUT:

[1,2]

PROBLEM STATEMENT:

Given an array of strings strs, group the anagrams together. You can return the answer in any order.

Example 1:

Input: strs = ["eat","tea","tan","ate","nat","bat"]

Output: [["bat"],["nat","tan"],["ate","eat","tea"]]

Explanation:

- There is no string in strs that can be rearranged to form "bat".
- The strings "nat" and "tan" are anagrams as they can be rearranged to form each other.
- The strings "ate", "eat", and "tea" are anagrams as they can be rearranged to form each other.

Example 2:

Input: strs = [""]

Output: [[""]]

Example 3:

Input: strs = ["a"]

Output: [["a"]]

Constraints:

- $1 <= strs.length <= 10^4$
- 0 <= strs[i].length <= 100
- strs[i] consists of lowercase English letters.

CODE:

```
import java.util.*;
class Solution {
    public List<List<String>> groupAnagrams(String[] strs) {
        Map<String, List<String>> map = new HashMap<>();
        for (String s : strs) {
            char[] ch = s.toCharArray();
            Arrays.sort(ch);
            String key = new String(ch);
            if (!map.containsKey(key)) {
                map.put(key, new ArrayList<>());
            }
```

```
            map.get(key).add(s);
        }
        return new ArrayList<>(map.values());
    }
}
```

INPUT:

strs = ["eat","tea","tan","ate","nat","bat"]

OUTPUT:

[["eat","tea","ate"],["bat"],["tan","nat"]]

PROBLEM STATEMENT:

– Frequency Queries

Problem

Maintain a collection of integers and support three operations:

1 x → Insert x

2 x → Delete one occurrence of x

3 f → Check whether any number occurs exactly f times

Print 1 if an element has frequency f, otherwise print 0.

CODE:

```
import java.util.*;
class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int q = sc.nextInt();
        Map<Integer, Integer> map = new HashMap<>();
        for (int i = 0; i < q; i++) {
            int op = sc.nextInt();
            int x = sc.nextInt();
            if (op == 1) {
```

```
            map.put(x, map.getOrDefault(x, 0) + 1);
        }
        else if (op == 2) {
            if (map.getOrDefault(x, 0) > 0) {
                map.put(x, map.get(x) - 1);
            }
        }
        else if (op == 3) {
            int found = 0;
            for (int freq : map.values()) {
                if (freq == x) {
                    found = 1;
                    break;
                }
            }
            System.out.println(found);
        }
    }
    sc.close();
  }
}
```

INPUT:

8

1 5

1 6

1 5

3 2

2 5

3 2

1 6

3 2

OUTPUT:

1

1

1

PROBLEM STATEMENT:

– Maximum Number of Distinct Elements in a Sliding Window

Problem

Given an array and a window size K, find the maximum number of distinct elements present in any contiguous window of size K.

CODE:

```
import java.util.*;

class Main {
   public static void main(String[] args) {
      Scanner sc = new Scanner(System.in);
      int n = sc.nextInt();
      int[] arr = new int[n];
      for (int i = 0; i < n; i++) {
         arr[i] = sc.nextInt();
      }
      int k = sc.nextInt();
      Map<Integer, Integer> map = new HashMap<>();
      int max = 0;
      for (int i = 0; i < k; i++) {
         map.put(arr[i], map.getOrDefault(arr[i], 0) + 1);
      }
      max = map.size();
      for (int i = k; i < n; i++) {
```

```
            int remove = arr[i - k];
            map.put(remove, map.get(remove) - 1);
            if (map.get(remove) == 0) {
                map.remove(remove);
            }
            map.put(arr[i], map.getOrDefault(arr[i], 0) + 1);
            max = Math.max(max, map.size());
        }
        System.out.println(max);
        sc.close();
    }
}
```

INPUT:

7

1 2 1 3 4 2 3

4

OUTPUT

4

PROBLEM STATEMENT:

Smart Traffic Navigation

Problem

A city has N junctions and M roads. Each road connects two junctions.

Using a nested collection, determine whether a route exists from a source junction to a destination junction.

CODE:

```
import java.util.*;
class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
```

```
int n = sc.nextInt();
int m = sc.nextInt();
ArrayList<ArrayList<Integer>> graph = new ArrayList<>();
for (int i = 0; i < n; i++) {
    graph.add(new ArrayList<>());
}
for (int i = 0; i < m; i++) {
    int u = sc.nextInt();
    int v = sc.nextInt();
    graph.get(u).add(v);
    graph.get(v).add(u);
}
int source = sc.nextInt();
int destination = sc.nextInt();
boolean[] visited = new boolean[n];
Queue<Integer> queue = new LinkedList<>();
queue.add(source);
visited[source] = true;
while (!queue.isEmpty()) {
    int current = queue.poll();
    if (current == destination) {
        System.out.println("YES");
        return;
    }
    for (int next : graph.get(current)) {
        if (!visited[next]) {
            visited[next] = true;
            queue.add(next);
        }
```

```
            }
        }
        System.out.println("NO");
        sc.close();
    }
}
```

INPUT:

5 4

0 1

1 2

2 3

3 4

0 4

OUTPUT:

Yes

PROBLEM STATEMENT:

Given an array of strings words and an integer k, return *the* k *most frequent strings*.

Return the answer sorted by the frequency from highest to lowest. Sort the words with the same frequency by their lexicographical order.

Example 1:

Input: words = ["i","love","leetcode","i","love","coding"], k = 2

Output: ["i","love"]

Explanation: "i" and "love" are the two most frequent words.

Note that "i" comes before "love" due to a lower alphabetical order.

Example 2:

Input: words = ["the","day","is","sunny","the","the","the","sunny","is","is"], k = 4

Output: ["the","is","sunny","day"]

Explanation: "the", "is", "sunny" and "day" are the four most frequent words, with the number of occurrence being 4, 3, 2 and 1 respectively.

CODE:

```
import java.util.*;
class Solution {
    public List<String> topKFrequent(String[] words, int k) {
        Map<String, Integer> map = new HashMap<>();
        for (String s : words) {
            map.put(s, map.getOrDefault(s, 0) + 1);
        }
        List<String> list = new ArrayList<>(map.keySet());
        Collections.sort(list, (a, b) -> {
            if (map.get(a).equals(map.get(b))) {
                return a.compareTo(b);
            }
            return map.get(b) - map.get(a);
        });
        return list.subList(0, k);
    }
}
```

INPUT:

words =["i","love","leetcode","i","love","coding"]

k = 2

OUTPUT:

["i","love"]

PROBLEM STATEMENT:

Student Marks Indexing System

Problem

A university stores student records containing:

Student ID

Student Name

Marks

Given multiple queries containing a student ID, efficiently retrieve the corresponding student's name and marks.

The solution must avoid scanning all student records for each query

CODE:

```
import java.util.*;

class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();
        Map<Integer, String> names = new HashMap<>();
        Map<Integer, Integer> marks = new HashMap<>();
        for (int i = 0; i < n; i++) {
            int id = sc.nextInt();
            String name = sc.next();
            int mark = sc.nextInt();
            names.put(id, name);
            marks.put(id, mark);
        }
        int q = sc.nextInt();
        for (int i = 0; i < q; i++) {
            int id = sc.nextInt();
            if (names.containsKey(id)) {
                System.out.println(names.get(id) + " " + marks.get(id));
            } else {
                System.out.println("Student not found");
            }
        }
```

```
        sc.close();
    }
}
```

INPUT:

3

101 Ravi 85

102 Priya 90

103 Arun 78

2

102

103

OUTPUT:

Priya 90

Arun 78

PROBLEM STATEMENT:

Product Inventory and Low-Stock Analysis

Problem

A supermarket maintains product information:

Product ID

Product Name

Stock

Price

Perform the following:

1. Update stock after sales.
2. Find products whose stock is below a given threshold.
3. Display products sorted by remaining stock.
4. Efficiently retrieve products using their ID.

CODE:

```
import java.util.*;
class Product {
    int id, stock;
    String name;
    double price;
    Product(int id, String name, int stock, double price) {
        this.id = id;
        this.name = name;
        this.stock = stock;
        this.price = price;
    }
    public String toString() {
        return id + " " + name + " " + stock + " " + price;
    }
}
class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();
        HashMap<Integer, Product> map = new HashMap<>();
        for (int i = 0; i < n; i++) {
            int id = sc.nextInt();
            String name = sc.next();
            int stock = sc.nextInt();
            double price = sc.nextDouble();
            map.put(id, new Product(id, name, stock, price));
        }
        int id = sc.nextInt();
```

```
        int sold = sc.nextInt();
        if (map.containsKey(id)) {
            Product p = map.get(id);
            if (sold <= p.stock) {
                p.stock -= sold;
            }
        }
        int threshold = sc.nextInt();
        System.out.println("Low Stock Products:");
        for (Product p : map.values()) {
            if (p.stock < threshold) {
                System.out.println(p);
            }
        }
        ArrayList<Product> list = new ArrayList<>(map.values());
        list.sort((a, b) -> a.stock - b.stock);
        System.out.println("Products Sorted by Stock:");
        for (Product p : list) {
            System.out.println(p);
        }
        int searchId = sc.nextInt();
        System.out.println("Product Details:");
        if (map.containsKey(searchId)) {
            System.out.println(map.get(searchId));
        } else {
            System.out.println("Product not found");
        }
        sc.close();
    }
```

}

INPUT:

3

101 Pen 20 10.0

102 Book 5 50.0

103 Bag 15 200.0

101 12

10

102

OUTPUT:

Low Stock Products:

101 Pen 8 10.0

102 Book 5 50.0

Products Sorted by Stock:

102 Book 5 50.0

101 Pen 8 10.0

103 Bag 15 200.0

Product Details:

Book 5 50.0